FVGs + IFVG + OTE Indicator — Design Document

Oct 18, 2026 · @Francisco

Overview

“FVGs+IFVG+OTE5” is a Pine Script v6 overlay for TradingView. It draws fair value gaps (FVGs), detects inverse FVGs (IFVGs) when price closes through them, and flags the IFVGs that line up with an ICT Optimal Trade Entry (OTE, the 0.62–0.79 retracement of a leg) on the chart timeframe or on the next two higher timeframes.

Term	Meaning in this script
FVG	Three-candle gap, built from the candles at [1], [2], [3]. “Long” (bullish) when the low of [1] is above the high of [3]; “short” (bearish) when the high of [1] is below the low of [3]. The drawn zone is widened to candle bodies when a body gap exists (suspension blocks).
1PFVG	First presented FVG after the session open (09:30 New York, 09:00 Madrid for DAX), tracked on minute charts up to `fpMaxTf` [60] (above 1m from the 1m candles: with `fp1m`, and always above 5m). Drawn in its own colors and extended to the right.
1PFVG inverse	The first FVG after the 1PFVG with the opposite bias (optional).
IFVG	An FVG that price closes through. A long FVG closed below its low becomes a bearish IFVG (“IFVG-”); a short FVG closed above its high becomes a bullish IFVG (“IFVG+”).
OTE	Retracement band between oteMin and oteMax (0.62 / 0.79 by default) of a leg. An IFVG is an “OTE IFVG” when price rejected the OTE of the leg before the FVG, or when it retraces into an IFVG that sits in the OTE of the leg since the inversion.
C.E.	Consequent encroachment: the midpoint of an FVG or of a candle wick.

What it produces:

- FVG boxes (or no-fill boxes, or high / low / C.E. segments), with mitigated FVGs closed at the bar that mitigated them.
- IFVG candidates and confirmed IFVGs, drawn on the FVG’s own box and labelled “IFVG+”, “IFVG-” or “IFVG± OTE <timeframes>”.
- OTE bands drawn to the right of the last candle, one column per tracked timeframe.
- Wick C.E. segments, optional signal markers and alerts.

Execution model

The script runs once per bar, top to bottom, and keeps all state in `var` globals and in arrays of user-defined types. Everything that confirms something (IFVG confirmation, signals) is evaluated on closed bars only (`barstate.isconfirmed`).

Per-bar pipeline, in source order:

1. **Inputs and constants.** Draw-mode flags (`fponly`, `ifvgOnly`, `hideReg`), filter flags (`fltOff`, `fltMid`) and the fixed filter values `flN = 1` and `flWin = 25`.
2. **Wick C.E.** If enabled, the wick of candle [1] is measured and a C.E. segment is drawn; the oldest is deleted beyond `nwicks`.
3. **Session setup.** `tz = Auto` resolves to Madrid (DAX) or New York; the 1PFVG time is parsed into `fph` / `fpm`.
4. **FVG detection.** A long and a short FVG are computed from candles [1]–[3]. Each new FVG becomes an `FVG` object and is classified: 1PFVG, 1PFVG inverse, or regular. Regular FVGs go through `route()` (merge and filter); in "1PFVGs" draw mode they are discarded.
5. **Sweep detection.** Short-term pivots (`flL`, `flR`) are tracked; a bar breaking the last one calls `onSweep()`, which reveals pending FVGs.
6. **FVG update.** `updfvgs()` creates or updates boxes, lines and C.E. lines, closes mitigated FVGs and fires bounce alerts, for the long, short and first-presented lists.
7. **Pivots for OTE.** Chart-timeframe pivots (`pivL`, `pivR`) and, through `request.security`, pivots of the next two timeframes on the ladder.
8. **IFVG candidate selection.** Close mode: every new long / short FVG replaces the bearish / bullish candidate. Swing mode: a significant swing pivot picks the last FVG before it. The candidate's FVG box is adopted as the IFVG box (`adopt`).
9. **Confirmation and cancellation.** A close through the FVG confirms the IFVG, labels it and starts the leg measurement; a close back through it cancels. Close mode signals on the confirming bar; swing mode signals on a later retest.
10. **Kept IFVGs.** `updKept()` extends kept boxes and restores invalidated ones.
11. **OTE step.** `gStep()` runs the three trackers of each of the three groups (`gBear`, `gBull`, `gSw`) and returns an OTE signal direction.
12. **Output.** OTE labels and highlight, `plotshape` markers, then the alerts.

Inputs

The settings dialog has seven sections, ordered so that what to draw and which FVGs to keep come first. Inputs that share an `inline` id sit on one line. Defaults are in brackets.

Section	Inputs	Effect
Enable / Draw / Filter	`dofp` 1PFVGs [on], `doifvg` IFVGs [on], `dowick` Wick C.E. [on]; `drawMode` [all]; `fltMode` [none]; `fpMode` 1PFVG filter [all]	`drawMode` is all, 1PFVGs, IFVGs or 1PFVGs+IFVGs. The IFVG modes hide regular FVGs, wicks, swing labels and signals; alerts keep working. `fltMode` is none, sweeps, sweeps&mid, relevant or recent relevant. `fpMode` is all, week (Mondays) or week+fridays.

Section	Inputs	Effect
FVG Settings	`visible` Historic compute [off], `dofvgalert`; `minfvg` [3], `maxfvg` [10000], `maxfvgs` keep [200]; `dupTol` merge tolerance [10%], `flL` / `flR` sweep pivots [3 / 3]; `drawStyle` [Box]; C.E. height / width / style; FVG colors	Size limits are in points. `dupTol = 0` turns merging off. `drawStyle` is Box, Box no fill, or Segments (hi / lo / C.E.).
1st Presented FVGs	`fpfvgtime` [0930], `tz` [Auto], `nfp` keep [6], `fp1m` from 1m candles [on]; day / week / Friday / inverse colors; `doifp` [off]	Session open for the 1PFVG; the week variant (Monday) has its own color.
Wick Settings	`nwicks` [30], `minwick` [20], `wickdx` [0 = auto], style, color, `dowickalert`	Wick C.E. segments and their bounce alerts.
IFVG Settings	Alerts, signals, labels; label parts `lblTf` OTE TF, `lblSign` + / -, `lblAp` A+ [all on]; `keepIFVG` + `keepMax` [on / 20]; `deConf` IFVG confirmation [Close]; `deInv` inversion rules [off], `deMaxInv` [1], `deDead` invalidated IFVG [Fade] + color; `ifvgswings` [off], significant swing, swing labels; `pivL` / `pivR` [10 / 3]; `atrLen` [14], `minATR` [4]; IFVG colors, transparency, text size and alignment	`ifvgswings` switches from the close-mode logic to the swing logic. `pivL` / `pivR` also set the pivots that find the leg before the FVG for OTE.
IFVG + OTE	`oteFvg` [on], `useOTE`, `oteOnly`, `showOTEsig`; `oteMin` / `oteMax` [0.62 / 0.79], `oteMode` [Overlap]; prior-leg rejection (`oteRej`, `oteRejAny`, `oteRejCap`); `oteHtf1`, `oteHtf2`, `oteHtfL` / `oteHtfR` [5 / 2]; band draw, offset [5], width [8]; band, border and text colors	`oteMode` is Overlap, Fully inside or C.E. inside. The band is drawn `oteOffset` candles right of the last candle, plus `col * (oteWidth + 1)` for its timeframe column, and is `oteWidth` candles wide.
Delivery Engine (Nuno)	`deDisp` displacement FVGs only [off], `deDispType` [Open to Close], `deStdLen` / `deStdX` [100 / 4], `deDispBar` [off] + color, `deFlag` + `deFlagW` [off / 3]; `deAplus` A+ setups [off], `deLiqLen` / `deSeqWin` [5 / 50], `deMssLen` [3], `deMssX` / `deMssWin` [2 / 2], `deKz` [on], `deHtfOn` + `deHtf` [off / 60]	Features of Nuno's Delivery Engine V20. See "Delivery Engine features".

Two values are not inputs but globals next to the filter inputs: `flN = 1` (FVGs shown after a sweep) and `flWin = 25` (the mid-arm window, as % of the arm length on each side of its middle).

Data structures

Five user-defined types carry the state. Pine cannot reassign globals inside functions, so every function mutates object fields instead.

FVG

One object per detected FVG, whether or not it is drawn.

Field	Role
`hi`, `lo`, `ce`	Zone and midpoint. Widened in place by merges.
`islong`	Bullish (long) FVG.
`start`, `end`	Start time (time of candle [3]); `end` is the mitigation time, or `na` while open.
`sbar`	Bar index where the FVG starts, used for the OTE prior leg and the mid-arm filter.
`rect`, `celine`, `hiln`, `loln`	Drawing handles: box, C.E. line, high and low segments.
`isfp`, `iswfp`, `isffp`, `isifp`	1PFVG, week (Monday) 1PFVG, Friday 1PFVG, inverse 1PFVG.
`dup`	Older almost identical FVG that absorbed this one.
`kept`, `kdir`	The box is currently shown as an IFVG, and its direction (-1 bearish, 1 bullish).
`cand`	Current IFVG candidate, not yet released.
`own`	The box exists only because of the IFVG (the FVG itself is not drawn).
`shown`	The FVG is in the drawn lists.
`lblD`	Direction of the `B` / `M` / `BM` label (the tagging breaker or block).
`ote`, `oLo`, `oHi`	[OTEF] OTE FVG flag, and the leg low / high (origin and running extreme).

FltState

State of the sweep filter, held in the single object `fs`: `post` (FVGs shown since the last sweep, starts at 1,000,000 so nothing is shown before the first sweep), `stH` / `stL` (last unswept short-term high / low) and their bars `stHbar` / `stLbar`.

PivSet

Pivots of one timeframe: parallel arrays `hb` / `hp` (pivot-high chart bar and price) and `lb` / `lp` (pivot lows), a `tag` such as “5m”, and `htf`. At most 60 pivots are kept. Three instances exist: `pv0` (chart), `pv1` and `pv2` (next two higher timeframes). Higher-timeframe pivots are stored with the chart bar they map to.

OteTrack

Follows one IFVG against the pivots of one timeframe.

Field	Role
`dir`, `zlo`, `zhi`	Direction and IFVG zone.
`legHi`, `legLo`, `rext`	The leg since the inversion, and the running retrace extreme since the FVG.
`armed`, `fired`	IFVG confirmed; OTE signal already fired for this confirmation.
`ctime`, `cbar`	Confirmation time and bar.
`pHi`, `pLo`, `rejHit`, `rej`	Prior leg, whether the confirming candle rejected its OTE, and the band drawn for it.
`band`, `tag`, `sbar`	Retrace OTE band, timeframe label, FVG start bar.
`col`	Margin column of the tracker: 0 for the chart timeframe, then 1 and 2 for the enabled higher timeframes (set by `gStart`).

OteGroup

Three trackers (`t0` chart, `t1`, `t2`) on the same IFVG, plus `flagged` (once per IFVG) and `tags` (timeframes that matched, e.g. “5m+15m”). Three groups exist: `gBear` and `gBull` for close mode, `gSw` for swing mode.

Global state

Variable	Content
`lfvgs`, `sfvgs`	Drawn long and short FVGs, sorted by start.
`fpfvgs`	1PFVGs and inverse 1PFVGs (at most `nfp`).
`pendF`	FVGs created since the last sweep and not shown (at most 60).
`keptIf`	FVGs whose box is currently shown as an IFVG.
`bearifvg`, `bullifvg`, `bearconfirm`, `bullconfirm`	Close-mode candidates and their confirmation times.
`ifvgBox`, `ifvgBoxb`	Boxes of the bullish and bearish IFVG under watch.
`swFvg`, `ifstime`, `ifshi`, `ifslo`, `ifsdir`, `ifsconfirm`	Swing-mode IFVG under watch.
`wicks`	Wick C.E. lines.

Functions and methods

Pine requires every function to return a value, so functions that only mutate state end with `true`.

FVG detection and drawing

Function	Role
`isLongFVG`, `isShortFVG`	Gap tests: `l1 > h3` and `h1 < l3`.
`longFVGHL`, `shortFVGHL`	Return the drawn zone extended to bodies, plus the wick-based zone used for the size filter.
`isFPFVG`	True inside the 1PFVG window: hour = `fph`, minute beyond `fpm` plus two candles, minute chart up to 3m, and the 1m path (`fpLow`) off.
`clearlastfp`	Keeps at most `nfp` 1PFVGs; the oldest stops extending and is killed.
`updfvgs(array)`	Per FVG: closes it when mitigated, creates its box or segments and C.E. line, follows the right edge, fires bounce alerts, and trims the array beyond `maxfvgs`. To keep it cheap, a finished FVG whose drawings already end at its `end` (`updEnd`) is skipped, and the right edge of a live FVG is moved only on the last bar and in real time (on history it stays where it was drawn).
`bgColor`, `lnColor`, `endsAt`, `isDone`, `kill`, `alerts`	Methods on `FVG`: colors (with the hidden-by-draw-mode rule), end time, mitigation test, delete drawings, bounce alerts.
`bounces(p)`	True when the bar touches level `p` and closes away from it.

Filtering and merging

Function	Role
`route(FltState, FVG)`	Entry point for a regular FVG: merge into a duplicate, or show it, or put it in `pendF`.
`findDup(FVG)`	Most recent drawn FVG (any bias) within the merge tolerance, searching `lfvgs`, `sfvgs`, `fpfvgs` and `keptIf`.
`mergeZone(g, f)`	Widens both FVGs to max high / min low, moves the C.E., updates box and lines, and re-opens `g`. Frozen when `g` is a kept IFVG.
`insertSorted(array, FVG)`	Inserts by start time and marks the FVG as shown.
`reveal(FVG)`	Shows a pending FVG, or merges it into an existing duplicate (the merged box then starts at the earlier start).
`onSweep(armStart)`	Reveals the last pending FVG and, with `sweeps&mid`, the largest one near the middle of the arm; clears `pendF`.

IFVG box handling

Function	Role
`adopt(FVG, dir)`	Turns the FVG's box (or its duplicate's) into the IFVG box; creates one if the FVG has none. Returns the box.
`release(FVG, confirmed, ctime)`	Candidate replaced: keep the box as IFVG if confirmed and `keepIFVG`, otherwise restore it.
`restore(FVG)`	Back to a plain FVG: deletes an own box, or restores colors, width, text and extension.
`updKept()`	Extends kept boxes to now; restores those the price closed back through; drops dead entries.
`findFVG(time, array)`	Swing mode: last FVG starting before a swing time.

OTE

Function	Role
`tfName`, `tfstr`	Timeframe labels for tags and alerts.
`addHigh`, `addLow`, `lastHigh`, `lastLow`	Methods on `PivSet`.
`barOfTime(t)`	Chart bar index of the candle at time `t`, scanning back at most 4,900 bars.
`feedHtf`	Registers a new higher-timeframe pivot once, on the first chart bar after its bar closed.
`priorLeg(dir, sbar, set)`	Returns `[legHigh, legLow]` of the leg before the FVG.
`oteStart`, `oteConfirm`, `oteCancel`, `oteStep`	Tracker life cycle: start watching a candidate, confirm it, cancel it, evaluate one bar.
`gStart`, `gConfirm`, `gCancel`, `gStep`	The same actions applied to the three trackers of a group; `gStep` returns the signal direction and builds the tags.

Helpers

`getstyle`, `getsize`, `gethalign`, `getvalign` map input strings to Pine constants. `wdx` returns the wick segment width. `hmParse` splits "0930" into hour and minute. `timeat` reads a future-offset time.

FVG detection, filtering and life cycle

Detection

On every bar the script looks at candles [1], [2] and [3]. A gap is valid when its wick-based height lies between `minfvg` and `maxfvg`. The stored zone is the wick gap, widened to a candle body when a body gap exists next to it. The FVG starts at the time of candle [3] and `sbar = bar_index - 3`. The newest long and short FVGs of the bar are kept in `lastlongfvg` and `lastshortfvg`, because the close-mode IFVG logic uses them even when they are later hidden or merged.

Classification of a new FVG, in order:

1. If a kept 1PFVG exists today (`fpKept`) and the inverse option is on, the first FVG of the opposite bias is the inverse 1PFVG.
2. Otherwise, if no 1PFVG exists yet and the bar is in the 1PFVG window, it is the day's 1PFVG: `hasfp` is set, and if `fpPass(dayofweek)` accepts the day (`fpMode`: all; week = Mondays; week+fridays = Mondays and Fridays) it becomes a 1PFVG (`iswfp` on Mondays, `isffp` on Fridays, `fpKept`) and goes to `fpfvgs`. A rejected one falls through to step 3.
3. Otherwise it is a regular FVG: `route()` handles it, or it is discarded when `drawMode` is 1PFVGs.

Life cycle

An FVG is mitigated when a candle closes or opens beyond its far side (below `lo` for long, above `hi` for short). Its `end` is then fixed and the box stops growing. Each array is trimmed to `maxfvgs` by deleting the oldest. 1PFVGs and inverse 1PFVGs extend to the right until `clearlastfp` retires them.

1PFVG from the 1m candles. `fpLowTf` (simple: a minute chart above 1m and at or below `fpMaxTf`, and `fp1m` or above 5m) selects "1" for `request.security_lower_tf` (else the chart timeframe, unused), returning the time / open / high / low / close arrays of the 1m candles of the chart bar; `fpLow` adds `dofp`. With `fpLow`, `isFPFVG` is false and, on each confirmed chart bar, every 1m candle newer than `fpLastT` is pushed to a 4-candle buffer (`fpO` / `fpH` / `fpL` / `fpC` / `fpT`) and tested as candle [0] of the 1m rule: no 1PFVG yet (`hasfp`), its hour = `fph` and minute beyond `fpm` + 2; `longFVGHL` / `shortFVGHL` on the three candles before it with the `minfvg` / `maxfvg` size test (bullish first). The FVG starts at the time of candle [3] of the buffer, `sbar` is the chart bar, it gets `isfp` / `iswfp` / `isffp` (Monday / Friday in `tz`) and goes to `fpfvgs` (`clearlastfp`); on a day `fpPass` rejects, the 1m FVG only sets `hasfp` and is dropped. `hasfp` / `hasifp` are reset by the chart bars as before. With `doifp`, once there is a kept 1PFVG (`fpKept`) and no inverse one yet (`ifpWin`), the same 1m test in the `fph` hour (any minute) keeps only the opposite bias of the last 1PFVG, and the FVG gets `isifp` / `hasifp` and goes to `fpfvgs`; the chart FVGs no longer make inverse 1PFVGs while `fpLow` is on. NinjaTrader: a 1m series (`AddDataSeries`, BarsInProgress 6) is added in Configure only when needed (2m chart up to `FpMaxMinutes`, with `FpFrom1m` or above 5m), and `OnFp1mBar` applies the same test to each completed 1m bar. Tradovate: not available (no lower timeframe data).

Sweep filter

With `fltMode = none`, every regular FVG is drawn. With `sweeps` or `sweeps&mid`, a regular FVG is drawn only when it matters around a liquidity sweep:

- Short-term highs and lows are pivots with `flL` / `flR` bars. A sweep is a bar whose high is above the last unswept short-term high (or whose low is below the last short-term low), tested against levels known before that bar.
- FVGs that are not shown wait in `pendF`. On a sweep, `onSweep()` shows the last one (the FVG created just before the sweep) and resets `fs.post` to 0, so the next `flN = 1` FVG is shown at creation.
- With `sweeps&mid`, `onSweep()` also shows the largest pending FVG whose start lies within `flWin = 25`% of the arm length around the arm's middle. The arm runs from the last opposite short-term pivot to the sweep bar.
- Before the first sweep nothing is shown. 1PFVGs and the close-mode IFVG logic are not filtered.
- 1PFVGs (and inverse 1PFVGs) always keep their own look and zone, whatever the filtering: `mergeZone()` never widens a 1PFVG, `reveal()` never moves its start, `bw()` ignores the displacement flag, `deKill()` always restores it (never the dead color), and `deRecolor()` and the OTE highlight skip

boxes whose FVG is a 1PFVG (`isFp()`, which follows `dup`). Ports: Ninja `MergeZone`, `Bw`, `IsFp`, `DeKill`, `DeRecolor`; Tradovate `mergeZone`, `bw`, `isFp`, `deKill`, `deRecolor`.
- With `recent relevant` (`fltRecent`, also sets `fltRel`), FVGs older than `relMaxAge` = 1000 bars (by `sbar`) are shifted off the front of `relF` each bar before the loop below (NinjaTrader `RecentRelevant`, `RelMaxAge`; Tradovate `fltRecent` in `relStep`). This bounds the per-bar loop, the costliest part of `relevant` on long histories.
- With `relevant` (`fltRel`), routed FVGs also go to `relF` (fields `swp`, `ext`, `dbar`). `onSweep()` shows the first FVG after the sweep as usual, but marks the last one before it `swp` instead of showing it; it is shown when a close goes through it. Any FVG flagged OTE is shown. `relSmall()` (height ≤ `mince`) makes `bgColor` move the transparency halfway to 100 and `lnColor` drop the border. Filled FVGs go to `relS`; once the pv0 pivots are known, a small one is hidden (`relHide`) unless `hasPiv` finds a pv0 pivot in [sbar-1, sbar+3], it is `ext` (widened by `mergeZone`) or kept / a candidate. On the last bar, `relB` boxes are drawn for live FVGs within `minATR` × ATR of the close. NinjaTrader: `FvgFilter = Relevant`, `RelStep()` after `AddLow(pv0, ...)`. Tradovate: `fltMode` relevant, `relStep(j)` after `updAtr()`. In both ports the near-price boxes are refreshed on each closed bar, not on every tick.

Merging near-identical FVGs

A new FVG, of any bias, whose high and low are each within `dupTol`% of its height of an existing FVG is not drawn. The existing one absorbs it:

1. `findDup()` returns the most recent match among the long, short, first-presented and kept-IFVG FVGs.
2. `route()` sets `f.dup` and calls `mergeZone()`: both FVGs take the max high and min low, and the old box, lines and C.E. are moved. If the old FVG was mitigated it is re-opened.
3. The new FVG is not stored in the drawn lists, so it is never drawn, but it still feeds the IFVG logic: `adopt()` and `release()` redirect to `f.dup`.
4. The same check runs again in `reveal()` when a pending FVG is shown, because the zone may have been drawn in the meantime. If the revealed FVG is the older one, the merged box starts at its start time.
5. A kept IFVG is never widened: its zone is frozen, and the new FVG is simply not drawn.

IFVG logic

An IFVG goes through three states: candidate (under watch), confirmed (a close went through the FVG) and cancelled (a close came back through it). Two modes choose the candidate.

Close mode (default)

- **Candidate.** Each new long FVG becomes the bearish candidate `bearifvg`; each new short FVG becomes the bullish candidate `bullifvg`. The previous candidate of that side is released.
- **Older candidates [IFO].** A candidate released unconfirmed (and with no inversions) goes to `bearOld` / `bullOld` (at most 3, oldest dropped). On the confirmation of the candidate of that side (first inversion with the inversion rules), `ifoConfirm` adopts each one the same candle broke (`dnBreak` / `upBreak`), labels it `IFVG-` / `IFVG+` and releases it as confirmed; then the list is cleared. No signal, alert or OTE tracker. On each confirmed bar `ifoPurge` drops the ones closed through (they were mitigated without the candidate confirming). Cost: at most 3 checks per side per bar. Same in both ports (`IfoPush` / `IfoConfirm`, `ifoPush` / `ifoConfirm`).

- **Confirmation.** On a closed bar, `close < bearifvg.lo` confirms a bearish IFVG (“IFVG-”); `close > bullifvg.hi` confirms a bullish one (“IFVG+”). The confirming bar is also the signal bar (`sig = -1` or `1`).
- **Cancellation.** After confirmation, a close back above `bearifvg.hi` (or below `bullifvg.lo`) clears the confirmation and the label, and cancels the OTE trackers. The candidate stays under watch. [HOLD] The check is `ifvgOut(d, lo, hi, hold)`, which returns [cancel, hold]: a close past the zone by more than `ifvgTolT` × `syminfo.mintick` cancels; a close past it within that margin with a wick beyond the close starts a hold (`bullHold` / `bearHold` / `ifsHold` = bar index), kept while the closes stay within the margin and cancelling after `ifvgHold` bars; a close back inside clears the hold. A close past the zone with no wick cancels. A released candidate passes its hold on in `khold`, and `updKept` restores a kept IFVG only when `keptOut(f)` (the same check with `f.khold`) cancels it. NinjaTrader: `IfvgOut(d, lo, hi, ref hold)` with `InvalidationTicks` × `TickSize`. Tradovate: `ifvgOut(d, lo, hi, key)`; the tick size is not available, so it is taken as the smallest price step seen in the bars.

Swing mode

One IFVG is under watch at a time.

- **Candidate.** A pivot (`pivL` / `pivR`) is significant when its distance to the last opposite pivot is at least `atr * minATR`, or when the swing filter is off. A significant pivot high selects the last long FVG that starts before it (bearish IFVG); a pivot low selects the last short FVG (bullish IFVG). The swing price is the origin of the OTE leg.
- **Confirmation and cancellation.** Same close rules as close mode, evaluated on every closed bar.
- **Signal.** After confirmation, on a later bar, price returns into the IFVG: for a bearish IFVG `high >= ifslo` and `high < ifshi`; for a bullish one `low <= ifshi` and `low >= ifslo`.
- Candidates are taken from `lfvgs` / `sfvgs`, so in this mode the sweep filter and the merge also decide which FVGs can become IFVGs.

One box per IFVG

The IFVG never gets a second box next to its FVG; the FVG’s own box is restyled.

- `adopt()` styles the box as an IFVG: IFVG colors, border width 2, no extension, right edge at the current time. If the FVG has no box (hidden by a draw mode or the filter, or absorbed by a duplicate), a box is created on the FVG, or on its duplicate, and flagged `own`.
- If the FVG is also a 1PFVG or an inverse 1PFVG, the box keeps the 1PFVG colors, border width and right extension. Only the text changes.
- `release()` runs when the candidate is replaced. With `keepIFVG` on and the IFVG confirmed, the box stays an IFVG box, its FVG gets `end = confirmation time`, and it joins `keptIf`. Otherwise `restore()` returns the box to a plain FVG, or deletes it if it was `own`.
- `updKept()` runs every bar: it extends kept boxes to now, and restores a kept IFVG when a confirmed close goes back through it (above `hi` for bearish, below `lo` for bullish). When `keptIf` exceeds `keepMax + 3`, the oldest non-candidate is restored.

Labels are set where the state changes: “IFVG-” / “IFVG+” on confirmation, empty on cancellation, “IFVG± OTE <timeframes>” with the `oteHighlight` border on an OTE signal. The label color is transparent when `Labels` is off.

OTE logic

Each IFVG under watch is followed by three `OteTrack` objects: the chart timeframe and the next two higher timeframes. An OTE on any enabled timeframe flags the IFVG, once per IFVG, and the matching timeframes are listed in the label (for example “5m+15m”). Each tracker runs two independent checks.

Timeframe ladder

The ladder is 3m (or lower), 5m, 15m, 1h, 4h, D. The chart sits on the highest rung at or below its timeframe, and the next two rungs are HTF1 and HTF2 (absent at the top of the ladder). Higher-timeframe pivots come from `request.security` with `lookahead_on` and `[1]` offsets, so they only use closed higher-timeframe bars and do not repaint. A new pivot is registered once, when the pivot time changes, and `barOfTime()` maps it to a chart bar.

Check 1: rejection of the prior leg

When the IFVG is confirmed, the leg before the FVG is measured with `priorLeg()`:

- Bearish IFVG: the last pivot high before the FVG start, down to the lowest low up to the FVG start (a prior down leg).
- Bullish IFVG: the last pivot low before the FVG start, up to the highest high up to the FVG start (a prior up leg).

The OTE band of that leg is measured from the leg’s end: for a bearish IFVG, `pLo + oteMin * range` to `pLo + oteMax * range`. The confirming candle is a rejection when its wick (its high for bearish, its low for bullish) reaches into that band and the close ends back away from it. `oteRejAny` also counts the running extreme of any candle between the FVG and the confirmation. `oteRejCap` requires the wick not to pass the upper OTE level; otherwise it may go deeper unless it takes out the leg origin. On a hit, a dashed band is drawn and the signal fires on the confirming bar. The IFVG itself need not be inside the band.

Check 2: retrace into the IFVG

After confirmation, the leg is measured from its origin to the extreme reached since the inversion:

	Bearish IFVG	**Bullish IFVG**
Origin	Highest high since the FVG (close mode: `h4` and later bars; swing mode: the swing high)	Lowest low since the FVG (close mode: `l4`; swing mode: the swing low)
Extreme	Lowest low since confirmation	Highest high since confirmation
OTE band	`legLo + oteMin * r` to `legLo + oteMax * r`	`legHi - oteMax * r` to `legHi - oteMin * r`
Tap	`high >= zlo`	`low <= zhi`

On higher timeframes the origin is moved to the latest higher-timeframe pivot when that pivot is beyond the retrace extreme. The IFVG matches the band according to `oteMode`: any overlap, fully inside, or its midpoint inside. The signal fires when the IFVG matches and price taps back into it on a later bar than the confirmation, and not if the signal already fired for this IFVG. The extreme is updated after the bar is evaluated, so the bar being tested never defines its own leg.

OTE FVGs (`oteFvg`)

Plain FVGs are flagged with chart-timeframe swings only. `otePL` / `otePH` keep the last pivot low / high of `pivL` / `pivR` (with their bars). When an FVG is created, `oteInit` sets the leg: for a bullish FVG, `oLo` is the lowest low from the last pivot low at or before the FVG start to candle [1] (the FVG candles alone when there is none) and `oHi` the highest high of the three FVG candles; mirrored for a bearish FVG. In `updfvgs`, `oteFvgStep` runs once per bar for every live FVG that is not a merged duplicate: the band is `oHi - oteMax * r` to `oHi - oteMin * r` (bullish) or `oLo + oteMin * r` to `oLo + oteMax * r` (bearish), matched with `oteMode` as for IFVGs; on a confirmed bar where the FVG matches and price taps into it, `ote` is set (once). The extreme is extended after the evaluation. The box text is built by `fvgLbl`: `A+`, or the `B` / `M` / `BM` tag with the arrow of `lblD` (↑ / ↓), then `OTE`; it is used wherever a plain FVG box gets its text (creation, A+ upgrade, breaker / block tags, restore after an IFVG candidate), so a restored box keeps its tags.

Bands and signals

Bands are drawn right of the last candle, `oteOffset` candles away and `oteWidth` candles wide, and are moved every bar. `oteX0(t)` gives the first bar of a band: each timeframe gets its own column, `t.col * (oteWidth + 1)` further right, so the bands and tags of several timeframes do not overlap. A band is yellow while the IFVG matches the OTE and gray while it does not. When a group raises a signal, `oteSig` is -1 or 1, a diamond is plotted if `showOTEsig` is on, the IFVG label becomes “IFVG± OTE <timeframes>”, and the OTE alert fires.

Delivery Engine features

These come from Nuno’s Delivery Engine (DeliveryEngine.txt, V20, updated to V29) and are all off by default, except the confirmation mode, which defaults to the original close rule. The state is computed in a block placed before the FVGs are created, tagged `[DE]` in the code.

Displacement filter

`deRange` is the candle range, open to close or high to low. A candle is a displacement candle when `deRange` exceeds `deStdX` times the standard deviation of the last `deStdLen` ranges. With `deDisp`, an FVG is created only when its middle candle (`[2]` on the detection bar) is a displacement candle. This applies to drawn FVGs and IFVG candidates; 1PFVGs and the inverse 1PFVG candidate are exempt (the gate also lets through any FVG while the inverse 1PFVG is still possible, and the ones that do not become it need displacement before `route()`). When `deDispBar` is on [off], the middle candle of each displacement FVG is colored (offset -1, as in V29), with or without `deDisp`. With `deFlag` [off], FVGs whose middle candle is a displacement candle (`FVG.disp`) are drawn with border width `deFlagW` [3] (`bw`), also with or without `deDisp`.

Confirmation and inversion rules

`deConf` = Full body requires both open and close beyond the FVG to confirm an IFVG (`upBreak` / `dnBreak`), in close and swing mode. With `deInv` (close mode only), each later close through the zone inverts it again instead of cancelling the confirmation: inversion 1 is the IFVG, inversion 2 is an IFVG in the other direction labelled “×2”, and so on up to `deMaxInv`. The next close through invalidates it: `deKill` fades the box (`deDead` = Fade, the box stops growing) or restores the plain FVG. Each inversion raises the IFVG signal and alert in its direction. OTE is tracked only for the first inversion.

A+ setups

Swing liquidity is pivots of `deLiqLen` bars on each side, kept until price trades beyond them; `deLastBsl` / `deLastSsl` are the bars of the last sweeps. An IFVG is A+ when it is confirmed within `deSeqWin` bars after a sweep of the opposite liquidity (IFVG+ after a swing low is taken), inside a killzone (`deKz`, ICTlevels sessions in the 1PFVG time zone, ignored above 15m) and, with `deHtfOn`, with the structure of `deHtf` in the same direction (`deStruct`, last closed HTF candle). The chart structure uses its own swings of `deMssLen` bars on each side [3] (V29; set it to `deLiqLen` for the V20 behaviour). An MSS is a close beyond the last swing high / low against the structure with a displacement candle (`deMssX` standard deviations) in its direction among the last `deMssWin` candles. A break against the structure without displacement is a CHoCH; a later break in the same direction with displacement, within `deSeqWin` bars, is taken as the MSS. When an MSS meets the same A+ conditions, the FVGs in its direction born since the sweep, and the ones created in the next candles, are tagged A+ on their boxes (`FVG.aplus`).

Labels

`ifvgTxt` builds the IFVG labels: “IFVG” plus the sign (`lblSign`), the inversion count, “OTE” with its timeframes (`lblTf`) and “A+” (`lblAp`), and an up / down arrow when the IFVG’s FVG was tagged at a mitigation block of the label’s direction (`mbd`, from the FVG field `mbD` through `mbOf`). With all parts on, the labels read as before plus the A+ tag.

Breakers

Self-contained block after `updKept()`, marked `[BRK]` (it can be removed without touching the rest). Inputs in the **Breakers** group, just before the Delivery Engine group.

- **Swings:** `ta.pivothigh` / `ta.pivotlow` with `brkLen` (2) on each side, fed to `blkPivot`, which keeps the last three pivots as an alternating sequence (`bSqP` prices, `bSqB` bars, `bSqT` 1 high / -1 low; a pivot of the same kind as the last one replaces it only when more extreme).
- **Formation:** when the sequence changes and reads low, high, low with the second low below the first, the high is armed (`bUpLvl` / `bUpBar`; once per swing: `bUpDone`); high, low, higher high arms the low (`bDnLvl`). Any other pattern of that kind disarms it, and an armed swing older than `brkAge` bars is dropped. The first close beyond the armed level adds the breaker already `broken` (`bkBar` = that bar). Its zone (`blkZone(dir, k0, body, whole)`) is the up-close (bullish) / down-close (bearish) candle at the swing bar or among the 3 before it, high / low for breakers, body for mitigation blocks (`body` true), plus the consecutive same-color candles before it in *Whole sequence* mode (limited to 300 bars). `brkAdd` also stores `zb`, the bar of the last zone candle.
- **Type `Breaker`:** `dir`, zone `hi` / `lo`, `t` (time of the first zone candle), `broken`, and its drawing `bx` (box) / `ml` (mid line). `brkAdd` keeps the last `brkKeep` per direction.
- **Life cycle (confirmed bars):** an untagged breaker older than `brkAge` bars from `zb` is removed (`brkDrop`); after the break a pivot high above `hi` (bullish) / pivot low below `lo` (bearish) whose bar is at or after `bkBar` sets `swung`; after that, a close through the other side removes it (also before the swing) and a candle reaching back into the zone is a pullback.
- **Tagging (`brkTag`):** on a pullback, live FVGs of `lfvgs` (bullish breaker) or `sfvgs` (bearish), and the confirmed IFVG of the trade direction (`bullifvg` / `bearifvg` in close mode, `swFvg` in swing mode), that overlap the zone set the new FVG field `brk` and draw the breaker: a dashed box over the zone and a line at its mid, from `t` to the current bar, once (not extended). `tagged` sets `pb` (last pullback bar). A drawn breaker stays in the array: every candle reaching into the zone sets `pb` and moves

the box / mid line right edge to it (a second pullback after price left extends it), and a close out of the zone in the breaker direction sets `won`. It fails on a candle closing beyond its far side (close below `lo` bullish, above `hi` bearish; the drawing candle included): the drawing is deleted when not `won`, kept otherwise, and the entry removed. It is also removed (drawing kept) when no pullback came for `brkAge` bars after `pb`. Colors: `brkC(dir)` (`brkCol` bullish, `brkColDn` bearish). Line width input `brkW`. `brkFlag` / `mbFlag` color the zone candles at tagging: the zone functions also return `zt1` (time of the last zone candle, field `t1`), and for every candle from `t` to `t1` a wick line (width 1) and a body line (width 5) in the rectangle color are drawn and kept in the field `fl`, deleted with the rectangle. `brkShd` / `mbShd` draw the box grey with 80% transparency and no border instead of the dashed outline (mid line unchanged); either of `brkBox` / `brkShd` (`mbBox` / `mbShd`) turns the drawing on. `brkBox` turns the drawing off; the state then follows the new field `tagged` (first matching pullback seen) instead of the box. `brkTag(f, b, ib)` also marks them: with `ib` (the IFVG box: `ifvgBox` for `bullifvg` / `swFvg`, `ifvgBoxb` for `bearifvg`) the box text is set to bold (`text.format_bold`, kept when the text changes later); for a plain FVG (`ib` na) that is unmitigated, not a confirmed IFVG box (a `kept` one, or the `cand` box of a confirmed `bullifvg` / `bearifvg` / `swFvg`: `confIfvg`) and has no A+ text, the box text is set to `B↑` / `B↓` with the IFVG text size and alignment. So the box of an unconfirmed IFVG candidate (the last FVG in close mode, otherwise without text) shows it too, and `adopt` keeps an existing `B` / `M` / `BM` text when an FVG becomes a candidate. `lblD` is set on every plain FVG tag, so the label has its sign when a candidate box goes back to the plain FVG (`restore`).
- **Not copied** to ICTScalp (labels only, no effect on signals).

Mitigation blocks

Block marked `[MB]`, a copy of the `[BRK]` block placed right after it, with its own names (`MBlock`, `mbs`, `mPH` / `mPL`, `mbZone`, `mbAdd`, `mbTag`) and inputs (**Mitigation blocks** group after **Breakers**: `mbOn`, `mbMode`, `mbLen`, `mbKeep`, `mbBox`, `mbCol` orange, `mbColDn` fuchsia, `mbSty`, `mbW`).

- **Formation:** the only difference with breakers. Bullish: low, high, **higher** low arms the high (`mUpLvl`); bearish: high, low, **lower** high arms the low (`mDnLvl`), on its own sequence (`mSqP` / `mSqB` / `mSqT`, `mbLen`). The zone is the candle **body** (`blkZone(..., true, ...)`).
- **Life cycle and drawing:** as for breakers (`mbAge`, `pb`, `won`, `mbC(dir)` with `mbColDn`).
- **Ports:** NinjaTrader `BlkSeq`, `BlkPivot`, `BlkZone(dir, k0, body, ...)`, `BlkDetect`, `BlkUpdate` (properties `BreakerMaxAge` / `MitigationMaxAge`, `BreakerColorDown` / `MitigationColorDown`); Tradovate `blkPivot`, `zoneOf(dir, k0, body, seq)`, `zoneBlock` (`brkAge` / `mbAge`, `brkColDn` / `mbColDn`).
- **Tagging (`mbTag(f, b, ib, which)`):** a plain FVG gets `M↑` / `M↓` (same conditions as `B↑` / `B↓`), and `mbD` is set on it too; when both a breaker (`brk`) and a mitigation block (`mbD` != 0) tagged it, whichever tags second writes `BM↑` / `BM↓`. For the IFVG (`which` 1 `bullifvg`, -1 `bearifvg`, 2 `swFvg`) the FVG field `mbD` is set to the block direction and the box text is rebuilt with `ifvgTxt` (inversion count when `deInv`, OTE part when `useOTE` and the group is flagged, its tags and A+), then set to bold. Every other `ifvgTxt` call passes `mbOf(f)` of its FVG, so the arrow survives later text changes (OTE signal, inversions); it only shows when `mbD` matches the label direction.
- **Inheritance (`ifvgInherit`):** on confirmed bars, each confirmed IFVG (`bullifvg` / `bearifvg`, or `swFvg`) is compared with the FVGs of its direction (`lfvgs` for IFVG+, `sfvgs` for IFVG-) that overlap it and are live or ended at or after the IFVG's FVG start. One tagged by a breaker of that direction (new FVG field `brkD`, set by `brkTag`) makes the IFVG label bold; one tagged by a mitigation block of that direction (`mbD`, now set on every overlapping plain FVG, labelled or not) sets the IFVG's `mbD` and rebuilds its text with the arrow (`mbTxt`, shared with `mbTag`). To keep it cheap, the scan runs only when `tagSeq` changed since the last scan for that IFVG (`inhSeq`); `bumpTags()` increments it when an

FVG gets a breaker or mitigation block tag, is widened by a merge, or is drawn while already tagged.
- **Not copied** to ICTScalp.

Poor liquidity

Inputs group **Poor liquidity**, the last one: `plMode` auto / yes / no [auto]; `poorLiq` is yes, or auto with a nano future (`syminfo.description` containing “nano”, or root / ticker starting with NNQ). With `poorLiq`, `longFVGHL` / `shortFVGHL` skip the suspension block (volume imbalance) extension, so `hi` / `lo` are the wick gap; and on the detection bar, when the three-candle FVG of a side does not pass `minfvg`, two candles [2], [1] closing in that direction with a wick gap (`low[1] > high[2]` bullish, `high[1] < low[2]` bearish) make the FVG instead, starting at [2] (`fk` = 2 for `timeat` and `sbar`). Everything after (filters, 1PFVG, routing) is unchanged. The 1m 1PFVG path also loses the extension. NinjaTrader: `PoorLiquidity` (FioPoorLiq), auto from the master instrument description / name. Tradovate: `plMode`, auto from the contract text (`symbolText`, NANO / NNQ), resolved once per init.

Data symbol (`FVGS+IFVG+OTE+PL.txt` only, a TradingView copy of this script, shown as “FVGs+IFVG+OTE5 PL”; it also sets a global `max_bars_back` of 4050 and 5000 for `time`): `plSrc` auto / symbol / chart [auto] and `plSym` [CME_MINI:MNQ1!] give `dataSym`: auto uses `plMicro()`, the micro continuous future for a nano root (NNQ, NES, NYM, N2K, or another N root with “nano” in the description: N to CME_MINI:M1!, CBOT_MINI for YM), else the chart symbol; `plAlt` is true when `dataSym` is not the chart symbol. `[dO, dH, dL, dC]` come from `request.security(dataSym, timeframe.period, ...)` with gaps off and no lookahead, and replace `open` / `high` / `low` / `close` everywhere in the logic; `dTR` replaces `ta.atr` (`ta.rma(dTR, atrLen)`). The HTF requests (`deStruct`, the OTE HTF pivots) and the 1m `request.security_lower_tf` use `dataSym` with the built-in series. With `plAlt`, auto `poorLiq` is off. The ports keep the chart data.

Drawing, alerts, limits and known risks

Drawing

- **Styles.** Box; Box, no fill (border only); Segments (high, low and C.E. lines, no box).
- **Draw modes.** In the IFVG modes, regular FVGs are drawn fully transparent rather than deleted, so they keep feeding the logic. A 1PFVG or IFVG box stays visible where the mode asks for it.
- **Limits.** The script allows 500 boxes and 500 lines. Segments use two or three lines per FVG, so fewer FVGs stay visible in that style. A box that is drawn only for an IFVG counts against the same limit.

Alerts

Alert	Condition
Bounce on FVG hi, lo or C.E.	`dofvgalert`, price touches the level of an open FVG and closes away; once per bar.
Bounce on wick C.E.	`dowickalert`.
“LONG IFVG <tf>” / “SHORT IFVG <tf>”	`doifvgalert` and an IFVG signal; suppressed when `oteOnly` is on.
“LONG IFVG+OTE <tf>” / “SHORT IFVG+OTE <tf>”	`doifvgalert`, `useOTE` and an OTE signal.

Design decisions

- **One box per IFVG.** Adopting the FVG box avoids two boxes in the same place, at the cost of the extra state fields (`kept`, `cand`, `own`, `dup`) and the restore path.
- **Merged FVGs stay objects.** An absorbed FVG is not drawn but still feeds the IFVG logic, so IFVG detection does not depend on what is visible.
- **Fixed filter values.** One FVG before and after a sweep and a 25% mid window are constants, not inputs.
- **Swing-mode confirmation** is evaluated on every bar. The original code evaluated it only on pivot-low bars, which is why it was moved to the top level.

Known risks and limits

- Merging two FVGs when the target already hosts another IFVG makes them share one box.
- A kept IFVG box is deleted if its FVG is shifted out of the `maxfvgs` window.
- `findFVG()` returns the oldest FVG in the list when none starts before the swing time.
- `barOfTime()` scans back up to 4,900 bars and `priorLeg()` up to 4,000 bars, on each new higher-timeframe pivot or IFVG start; the script declares `max_bars_back = 5000`.
- Higher-timeframe OTE needs the two requested timeframes to be higher than the chart, so near the top of the ladder one or both trackers are absent.
- Historic compute is off by default, so FVGs older than the last `maxfvgs` are not shown when scrolling back.

Derived indicators

`tools/mkfvgderiv.py` generates `FVGS+IFVG+OTE4.txt` (“FVGs+IFVG+OTE4”) and `ICTblocks.txt` (“ICTblocks”) from `FVGS+IFVG+OTE.txt`; rerun it after every change to the source and never edit the outputs. The source carries comment markers: `//#cut T` … `//#end` lines are removed in derivative `T` (`ote4`, `blk`, or `sw` for both), and a `//#T code` line is code only in `T`. Inputs a derivative does not show become constants with their default value, or the value in the script tables.

- FVGs+IFVG+OTE4: cuts the Delivery Engine section (stubs: `deDispOK` false, no MSS bars), breakers, mitigation blocks and `ifvgInherit`, and the swing-mode IFVG code (`ifvgswings` false); it keeps `deKill` / `deRecolor` and the inversion rules. Inputs `brk*`, `mb*`, `de*` (except `deConf`, `deInv`, `deMaxInv`, `deDead`, `deDeadCol`), `ifvgswings`, `useSwingFilter`, `showSwing` and `lblAp` become constants (blocks off, A+ off).
- Both: with `deFlag` and `deAplus` constant off, `fold()` removes `bw()` (calls become 1) and `deAp()` (calls become false).
- ICTblocks: cuts the FVG drawing in `updfvgs` (a finished FVG is simply skipped) and the box in `adopt` (IFVG candidates have no box; the box setters on `na` do nothing), and the swing-mode IFVG code. Kept inputs: `brk*`, `mb*`, `deDisp`, `deDispType`, `deStdLen`, `deStdX`, `deDispBar`, `deDispCol`, `deConf`, `deInv`, `deMaxInv`, `plMode`, `minfvg`, `maxfvg`; block rectangles on by default, with a semi-transparent fill (85) and a B / M label with the arrow, placed by the added `brkLbl` / `mbLbl` inputs (default: right side, centered), and `brkNoFill` / `mbNoFill` (border only: transparency 100 instead of 85 / 93) (`BLK_SUBST`). ICTblocks-only lines (`//#blk`): `brkWait` / `mbWait` draw the block at its break (fill 93) and the loop extends it until the tag, which reuses the box (fill 85, right edge at the pullback); `brkDrop` / `mbDrop` erase an untagged box when the block is removed. 1PFVGs, wicks, OTE (and its higher timeframe requests), filters, A+, the displacement flag, signals and alerts are constants off.